

Reviewer Pack — Level-1 Fourier Mass Fraction of Term Family Bounds Monotone...

Entry 9ec4f1b18412 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 02:22:44 UTC

Level-1 Fourier Mass Fraction of Term Family Bounds Monotone CLIQUE DNF

Entry ID: 9ec4f1b18412 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 02:22:44 UTC

1. Conjecture

Field A (mathematical branch): Fourier-analytic hypercontractivity / KKL low-level concentration on syntactic term-incidence empirical measures **Field B** (complexity object): Monotone DNF leaf size $L(F)$ for the k -CLIQUE indicator F^*_v on K_v with $k = \lfloor \sqrt{v} \rfloor$, $n = v(v-1)/2$ edge variables (Razborov 1985 / Andreev / Alon-Boppana monotone regime)

Statement:

For a monotone DNF $F = \bigwedge_{i=1}^s T_i$ over n variables, view the term-support indicators $\{1\{T_i\}\}_{i=1}^s \subset \{0,1\}^n$ as an empirical sample and let $\hat{\mu}F(S) := (1/s) \sum_i (-1)^{|S \cap T_i|}$ be the Walsh-Fourier coefficient of the empirical measure at $S \subseteq [n]$. Define the level-1 low-frequency fraction $\lambda(F) := (\sum_{|S|=1} \hat{\mu}F(S)^2) / (\sum_{1 \leq |S| \leq 2} \hat{\mu}F(S)^2)$ and the complexity measure $\mu(F) := -\log_2 \max(\lambda(F), 1/n)$. Conjecture: (a) for every monotone DNF F of leaf size $L(F) = \sum_i |T_i|$, $\mu(F) \leq 2 \cdot \log_2(L(F)+2) + 4$; (b) for the k -CLIQUE indicator F^*_v on K_v with $k = \lfloor \sqrt{v} \rfloor$, $\mu(F^*_v) \geq (1/8) \cdot v$ for all $v \geq 4$. Equivalently, $\log_2 L(F^*_v) \geq \Omega(v) = \Omega(\sqrt{n})$, a monotone CLIQUE leaf-size lower bound for $k = \sqrt{v}$.

Rationale (proposer's reasoning):

By KKL/Friedgut, a Boolean-valued empirical measure with level-1 mass dominating its low-level spectrum behaves like a junta of few variables — exactly the regime of small DNFs whose terms touch few variables and whose Fourier signal sits on singletons. The k -CLIQUE indicator's term family is the $(v \text{ choose } k)$ edge-sets of k -cliques, an S_v -equivariant design that uniformizes level-1 character sums and pushes ≤ 2 -level mass onto edge-pair correlations, suppressing λ to $O(1/v)$. The measure is SYNTACTIC on the term list (two monotone DNFs computing the same Boolean function can have very different μ), so it is shielded from Razborov-Rudich, and uses $\log_2$ of an $L^{2/L}$ fraction over level-1 and level-2 characters — a non-ring functional with no F_q polynomial lift (Aaronson-Wigderson algebrization-safe).

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9d5f901c3f362d2a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds for ensemble (A) (random monotone DNFs, $s \in \{10, 20, 40, 80, 160\}$, widths 2-6, $n \in \{16, 24, 32, 40\}$) every sampled F must satisfy $\mu(F) \leq 2 \cdot \log_2(L(F)+2)+4$; for ensemble (B), every $v \in \{4, \dots, 8\}$ must satisfy $\mu(F^*_v) \geq v/8$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.88	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.82	SAFE	SAFE
ALGEBRIZATION	SAFE	0.82	SAFE	SAFE
KARP_LIPTON	SAFE	0.86	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - monotone DNF Fourier level-1 concentration clique lower bound - Walsh Fourier coefficients term incidence empirical measure monotone circuit complexity - KKL hypercontractivity monotone DNF leaf size k-clique Razborov

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=197.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def walsh_fourier_coefficient(F, S):
    n = len(F)
    sum_term = 0
    for i in range(2**n):
        term = (-1)**sum((i >> j) & 1 for j in range(n) if (j + 1) in S)
        sum_term += F[i] * term
    return Fraction(sum_term, 2**n)

def level_1_low_frequency_fraction(F):
    n = len(F)
    sum_level_1 = sum(walsh_fourier_coefficient(F, {j+1 for j in range(n) if (i >> j) & 1})**2 for i in range(2**n))
    sum_level_2_and_above = sum(walsh_fourier_coefficient(F, {j+1 for j in range(n) if (i >> j) & 2})**2 for i in range(2**n))
    return Fraction(sum_level_1, sum_level_1 + sum_level_2_and_above)

def complexity_measure(lambda_F):
    if lambda_F >= 1/len(F):
        return -math.inf
```

```

    return -math.log2(max(lambda_F, 1/len(F)))

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [16, 24, 32, 40]
    s_values = [10, 20, 40, 80, 160]
    widths = range(2, 7)

    results_A = []
    for n in n_values:
        for s in s_values:
            for width in widths:
                F = [random.choice([0, 1]) for _ in range(2**n)]
                L_F = sum(F)
                mu_F = complexity_measure(level_1_low_frequency_fraction(F))
                results_A.append((L_F, mu_F))

    results_B = []
    for v in range(4, 9):
        k = math.floor(math.sqrt(v))
        F_v = [0] * (2**v)
        for i in range(v):
            for j in range(i+1, v):
                F_v[(1 << i) | (1 << j)] = 1
        mu_F_v = complexity_measure(level_1_low_frequency_fraction(F_v))
        results_B.append((mu_F_v, v))

    mean_mu_A = sum(mu_F for _, mu_F in results_A) / len(results_A)
    std_mu_A = math.sqrt(sum((mu_F - mean_mu_A)**2 for _, mu_F in results_A) / len(results_A))
    support_fraction_A = sum(1 for _, mu_F in results_A if mu_F <= 2 * math.log2(L_F + 2) + 4) / len(results_A)

    all_results_B = [mu_F_v >= v/8 for mu_F_v, _ in results_B]
    support_fraction_B = sum(all_results_B) / len(all_results_B)

    conjecture_holds_A = support_fraction_A == 1.0
    conjecture_holds_B = support_fraction_B == 1.0

    counterexample_A = "" if conjecture_holds_A else "random DNF with  $\mu > 2 \cdot \log_2(L+2)+4$ "
    counterexample_B = "" if conjecture_holds_B else "k-CLIQUE indicator  $F^*_v$  with  $\mu < v/8$ "

    return {
        "metric_name": "complexity_measure",
        "metric_value_A": mean_mu_A,
        "metric_value_B": support_fraction_B,
        "instances_tested_A": len(results_A),
        "instances_tested_B": len(results_B),
        "conjecture_holds_A": conjecture_holds_A,
        "conjecture_holds_B": conjecture_holds_B,
        "counterexample_A": counterexample_A,
        "counterexample_B": counterexample_B
    }

if __name__ == "__main__":
    import sys
    seeds = [int(seed) for seed in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30))

```

```

results = []
for seed in seeds:
    trial = run_trial(seed)
    print(f"TRIAL: {trial}")
    results.append(trial)

mean_mu_A = sum(trial["metric_value_A"] for trial in results) / len(results)
std_mu_A = math.sqrt(sum((trial["metric_value_A"] - mean_mu_A)**2 for trial in results) / len(results))
support_fraction_A = sum(1 for trial in results if trial["conjecture_holds_A"]) / len(results)

all_results_B = [trial["conjecture_holds_B"] for trial in results]
support_fraction_B = sum(all_results_B) / len(all_results_B)

if support_fraction_A == 1.0 and support_fraction_B == 1.0:
    print(f"RESULT: SUPPORTED mean={mean_mu_A} std={std_mu_A} support_fraction={support_fraction_A}")
elif not all_results_B:
    print(f"RESULT: FALSIFIED counterexample=\"k-CLIQUE indicator F*_v with  $\mu < v/8$ \" first_failed_trial={first_failed_trial}")
else:
    print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8b489d7d.py", line 94, in <module>
    trial = run_trial(seed)
            ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8b489d7d.py", line 50, in run_trial
    mu_F = complexity_measure(level_1_low_frequency_fraction(F))
            ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8b489d7d.py", line 28, in level_1_low_frequency_fraction
    sum_level_1 = sum(walsh_fourier_coefficient(F, {j+1 for j in range(n) if (i >> j) & 1})*2 for i in range(n))
    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8b489d7d.py", line 28, in <genexpr>
    sum_level_1 = sum(walsh_fourier_coefficient(F, {j+1 for j in range(n) if (i >> j) & 1})*2 for i in range(n))
    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8b489d7d.py", line 23, in walsh_fourier_coefficient
    sum_term += F[i] * term
                ~^^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with an `IndexError` in `walsh_fourier_coefficient` before producing any `RESULT` line, so neither ensemble (A) nor (B) was evaluated. Per the rules, a crashed test yields `INCONCLUSIVE`. | next: Fix the indexing bug (likely a 0-based vs 1-based variable indexing mismatch between `F` and the subset construction `{j+1 ...}`) and re-run all 30 seeds plus $v \in \{4..8\}$.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	153245
2	preregistration	claude_max	opus	0	0	6741
3	novelty	claude_max	opus	0	0	4662
4	novelty	claude_max	opus	0	0	8103
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16152
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11693
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12646
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14182
9	judge	claude_max	opus	0	0	4582

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 232006 ms total latency. Provider mix: `{'claude_max': 5, 'ollama_remote': 4}`

(full prompt+response transcripts available in `research/audit/9ec4f1b18412.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9ec4f1b18412.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/9ec4f1b18412.tar.gz` (if generated)